Newton School of Technology

06

Lecture 06

CPU Scheduling Fundamentals

by - Brijesh Shukla

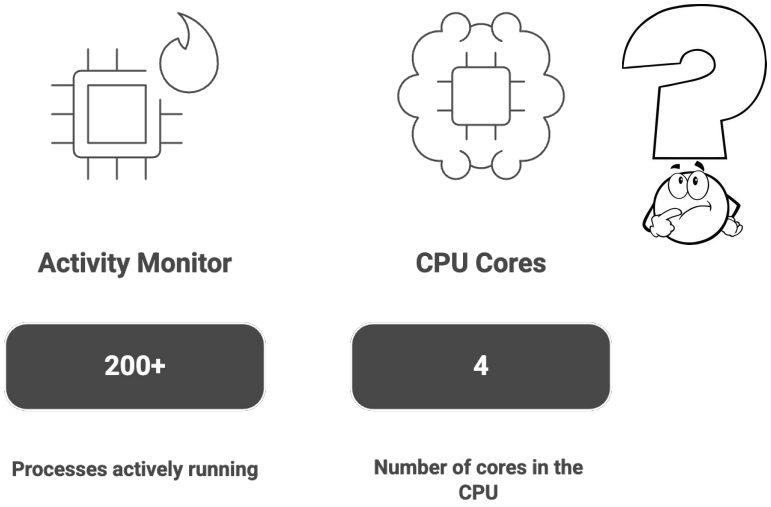
C07: Operating Systems

The Impossible Multitasking Trick

Newton School of Technology

- Your Mac has 4-8 CPU cores
- Activity Monitor shows 100+ processes running
- All apps feel responsive simultaneously
- Music plays, browser loads, code compiles. All at once!

Question: How is this possible?

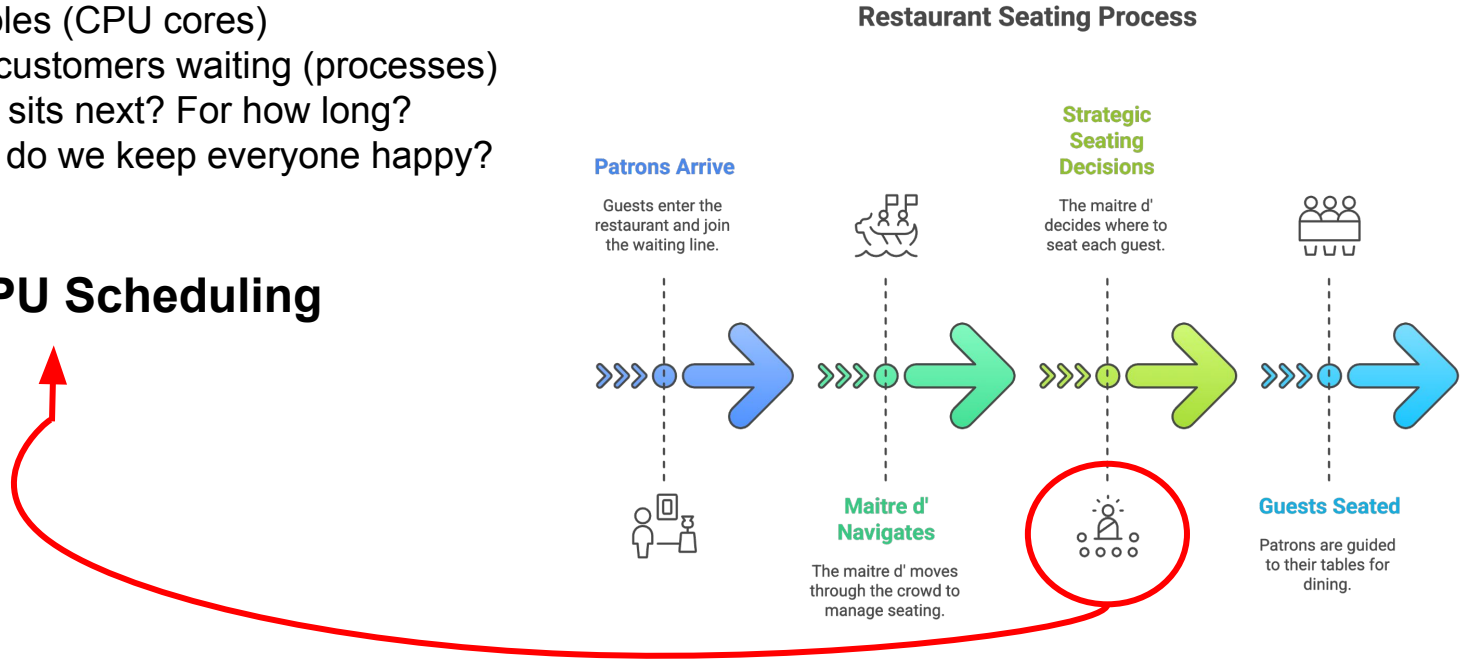


Too Many Guests, Not Enough Tables

Newton School of Technology

- 4 tables (CPU cores)
- 100 customers waiting (processes)
- Who sits next? For how long?
- How do we keep everyone happy?

This is CPU Scheduling



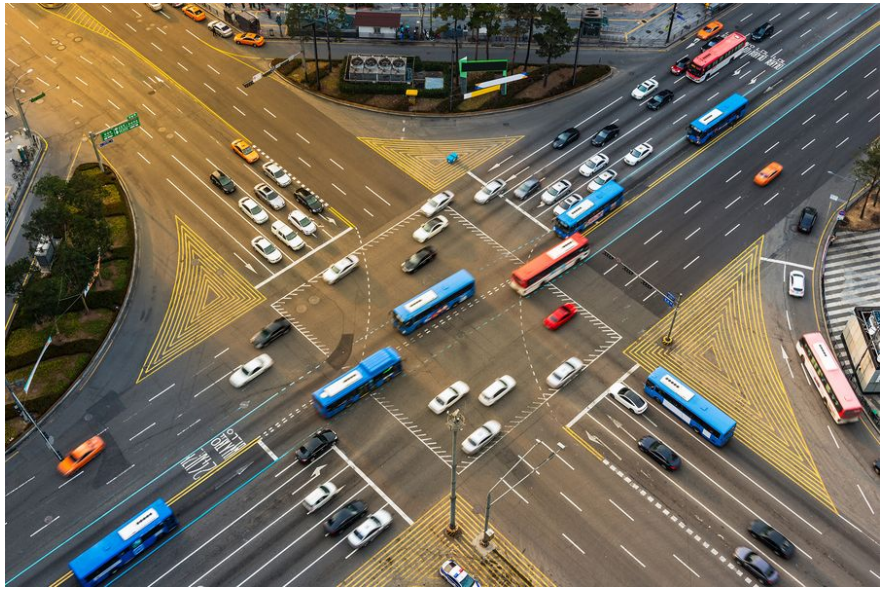
What is CPU Scheduling?

Newton School of Technology

The Operating System's Traffic Controller

- Decides which process runs next
- Allocates CPU time fairly
- Maximizes system performance
- Keeps interactive apps responsive

Executes 10-100 times per second!



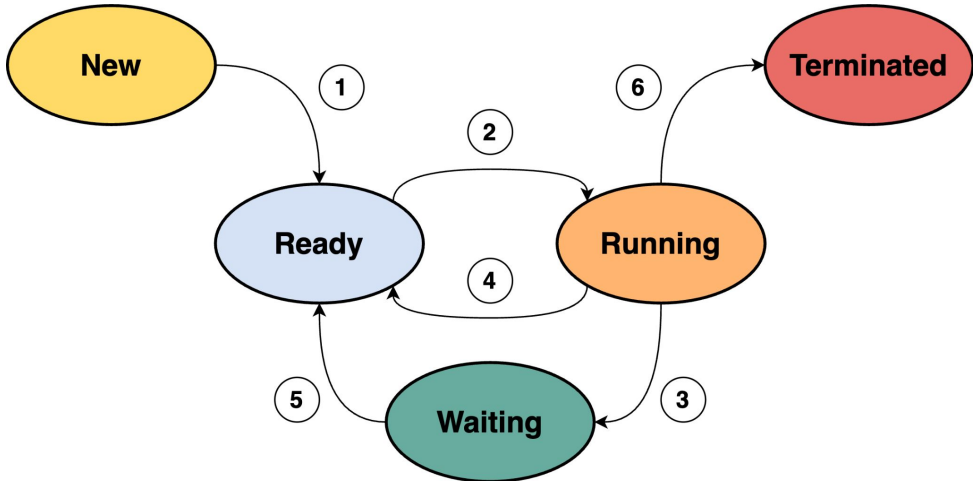
When Scheduling Happens

Newton School of Technology

Four Critical Moments

- **Process blocks** → Waiting for I/O
- **Process finishes** → Terminates completely
- **Time expires** → Quantum runs out (preemption)
- **I/O completes** → Returns to ready state

Each moment: Scheduling decision needed



What Makes Good Scheduling?

How to balance competing goals in CPU scheduling?

Maximize CPU Utilization

Keeps the CPU busy, ensuring no idle time.

Maximize Throughput

Completes more work in a given time.

Minimize Turnaround Time

Finishes processes faster, reducing overall execution time.

Minimize Waiting Time

Reduces queueing, making processes start sooner.

Minimize Response Time

Provides a snappy feel, enhancing user experience.

Problem: Can't optimize all simultaneously!

CPU Utilization

Newton School of Technology

CPU Utilization = (Busy Time / Total Time) × 100%

Example:

- 100 seconds total
- 80 seconds busy, 20 seconds idle
- Utilization = 80%

Goal: 40-90% (100% unrealistic)

Throughput & Turnaround Time

Newton School of Technology

How Much Work Gets Done

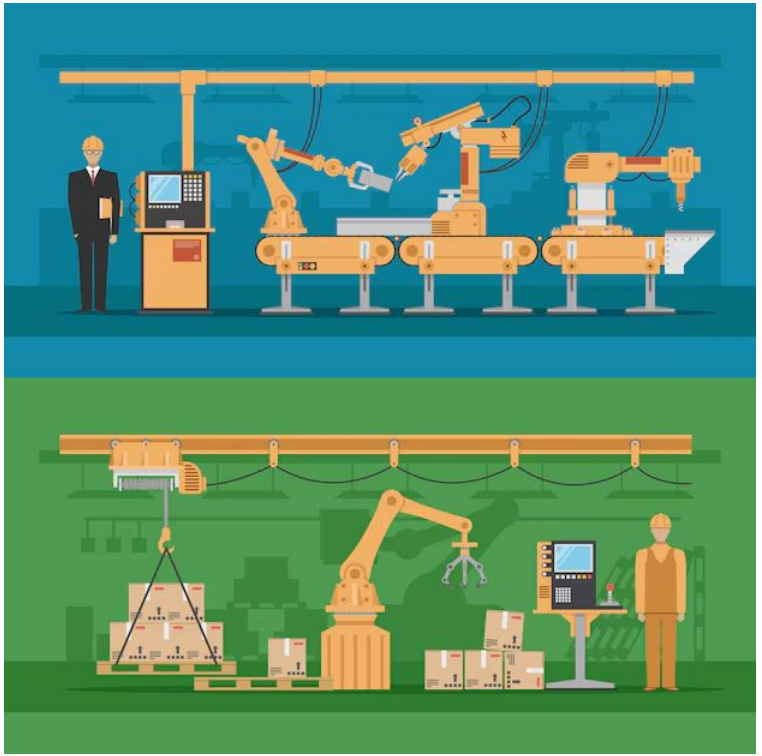
Throughput: Processes completed per second

- Higher = better

Turnaround Time: Arrival → Completion

- Lower = better
- **Formula:** Completion Time - Arrival Time

Trade-off: Fast completion vs total work



Waiting Time & Response Time

Newton School of Technology

The User Experience Metrics

Waiting Time: Time spent in ready queue

- What scheduler directly controls

Response Time: Arrival → First CPU time

- **Critical for interactive apps!**
- Difference between "snappy" and "sluggish"

Users notice 100ms delays

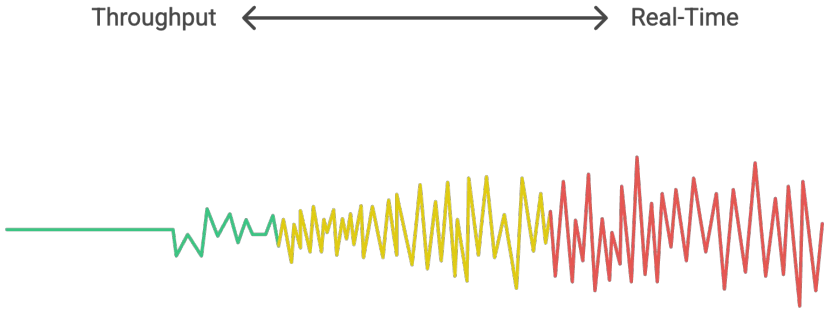
Batch vs Interactive Systems

Newton School of Technology

System Type	Priority Goals
Batch (servers)	Throughput, CPU utilization
Interactive (laptop)	Response time, fairness
Real-Time (aircraft)	Meet deadlines, predictability

Your Mac: Optimized for response time

System priority goals range from throughput to real-time deadlines.



Servers

Maximize throughput and CPU utilization

Laptops

Balance response time and fairness

Aircraft

Meet deadlines with predictability

Preemptive vs Non-Preemptive

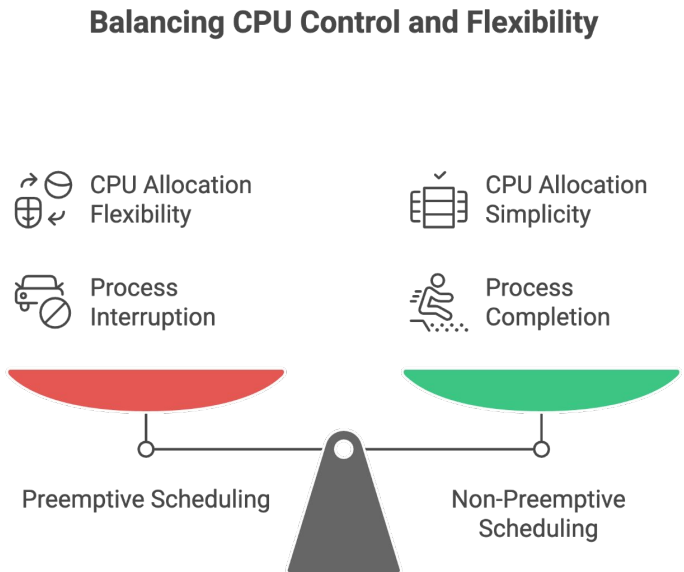
Newton School of Technology

Preemptive:

- OS can forcibly take CPU away
- ✅ Fair sharing, good response time
- ✅ All modern systems use this

Non-Preemptive:

- Process runs until it finishes or blocks
- ❌ One long process monopolizes CPU
- Simple but poor for interactive



Preemption's Secret Weapon

Newton School of Technology

Time Quantum (Time Slice):

- Each process gets fixed CPU time
- Typical: 10-100 milliseconds
- Too large → behaves like non-preemptive
- Too small → excessive overhead

Timer interrupt forces switch

The Dispatcher

Newton School of Technology

Dispatcher = The Mechanism

1. Save current process state
2. Load next process state
3. Switch to user mode
4. Jump to new instruction

Dispatch Latency: 1-5 microseconds

- Must be FAST (runs constantly)

Scheduling Queues

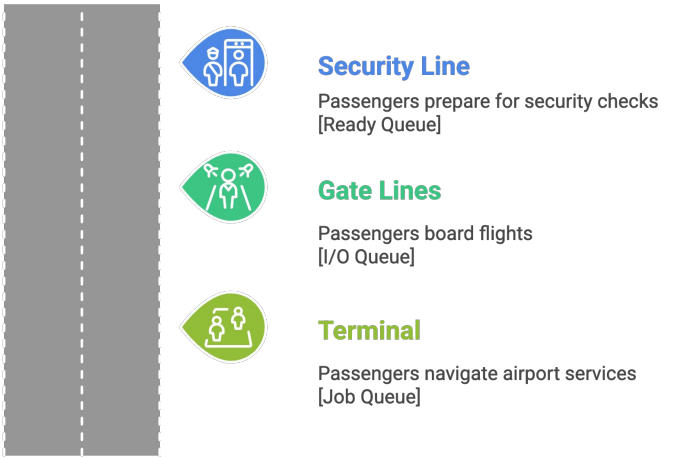
Newton School of Technology

Three Key Queues:

- **Ready Queue:** Waiting for CPU (most important)
- **I/O Queues:** Waiting for devices (disk, network)
- **Job Queue:** All processes in system

Processes move between queues constantly

Airport Passenger Flow as Scheduling System



CPU-Bound vs I/O-Bound

Newton School of Technology

Processes Alternate: [CPU burst] → [I/O burst] → [CPU burst] → [I/O]...

CPU-Bound:

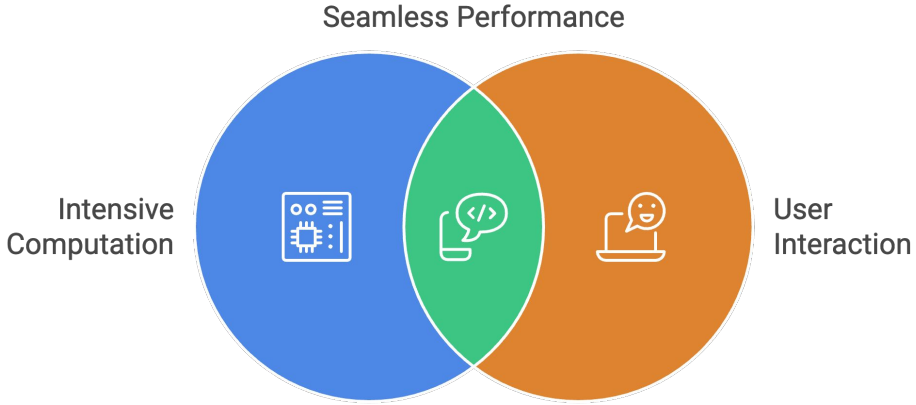
- Long CPU bursts (100ms+)
- Video encoding, compilation, simulations
- Needs: Long quantum, throughput

I/O-Bound:

- Short CPU bursts (1-10ms)
- Text editors, browsers, terminals
- Needs: Quick response, high priority

Observation: Most CPU bursts are SHORT

- Many 1-10ms bursts
- Few 100ms+ bursts



This pattern drives scheduling design

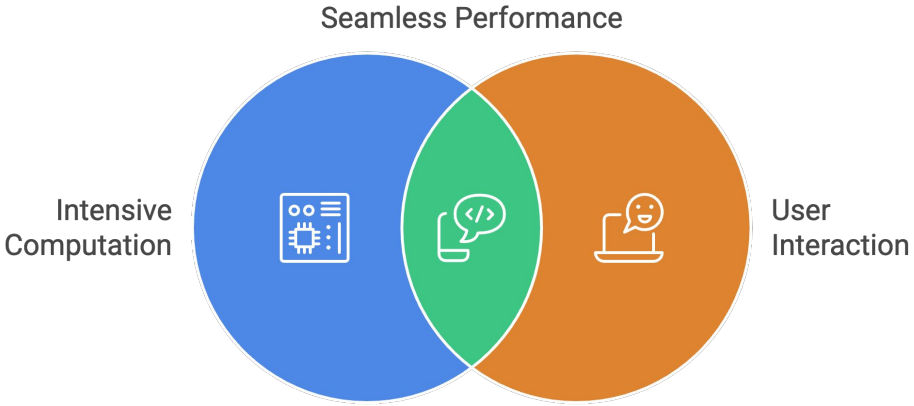
The Perfect Mix

Newton School of Technology

Best System Performance:

- Mix of CPU-bound AND I/O-bound
- CPU-bound keeps processor busy
- I/O-bound keeps devices busy
- **Overlap:** I/O happens while CPU computes

Your Mac automatically balances this



Five Ways to Schedule

1. **FCFS:** First-Come, First-Served (simple, fair arrival)
2. **SJF:** Shortest Job First (optimal avg wait)
3. **Round Robin:** Time slices (fair, responsive)
4. **Priority:** Important first (real-time)
5. **Multilevel:** Combination (modern systems)

No universal "best"

Each optimizes different goals

FCFS – First Come, First Served

Newton School of Technology

How it works: Queue, process in arrival order

Pros: ✅ Simple, ✅ No starvation

Cons: ❌ Poor wait time, ❌ Convoy effect

Convoy Effect: Short process waits behind long one

Example: P1(24) → P2(3) → P3(3)

- P2 waits 24 seconds for 3-second job!

SJF – Shortest Job First

Newton School of Technology

How it works: Execute shortest burst first

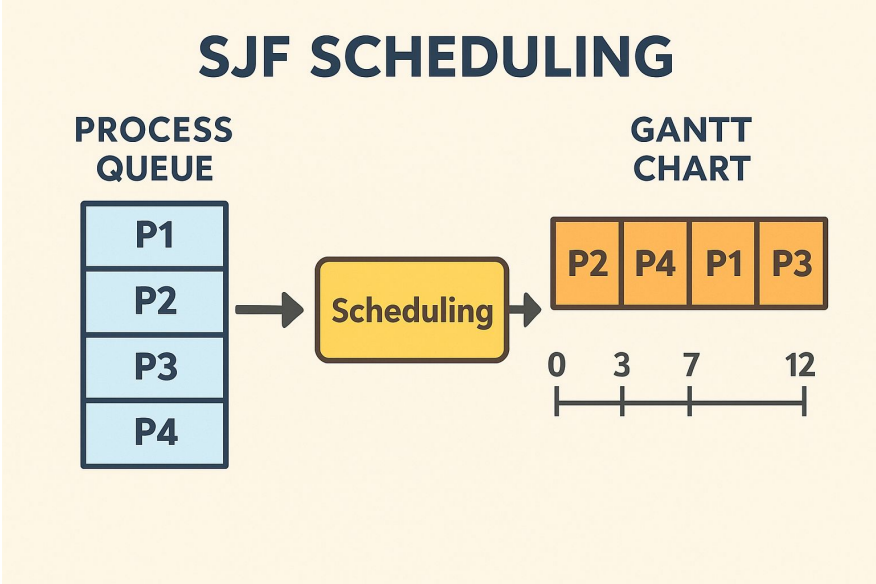
Pros: ✅ Optimal average waiting time

Cons: ❌ Need burst prediction, ❌ Starvation

Same processes: P2(3) → P3(3) → P1(24)

- Average wait: 3 vs 17 (FCFS)

Problem: How to predict next burst?



Round Robin

Newton School of Technology

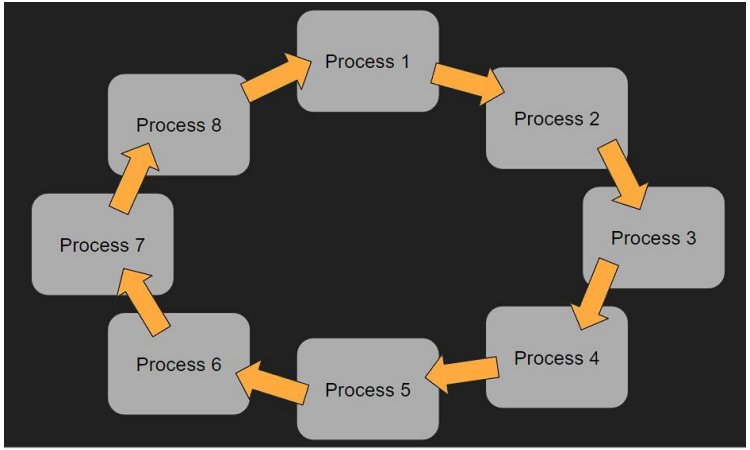
How it works: Each gets small time slice (quantum)

Pros: ✅ Fair, ✅ Good response, ✅ No starvation

Cons: ❌ Higher turnaround, ❌ Context switches

Perfect for interactive systems!

Quantum = 4: P1 → P2 → P3 → P1 → P1... (rotating)



Priority Scheduling

Newton School of Technology

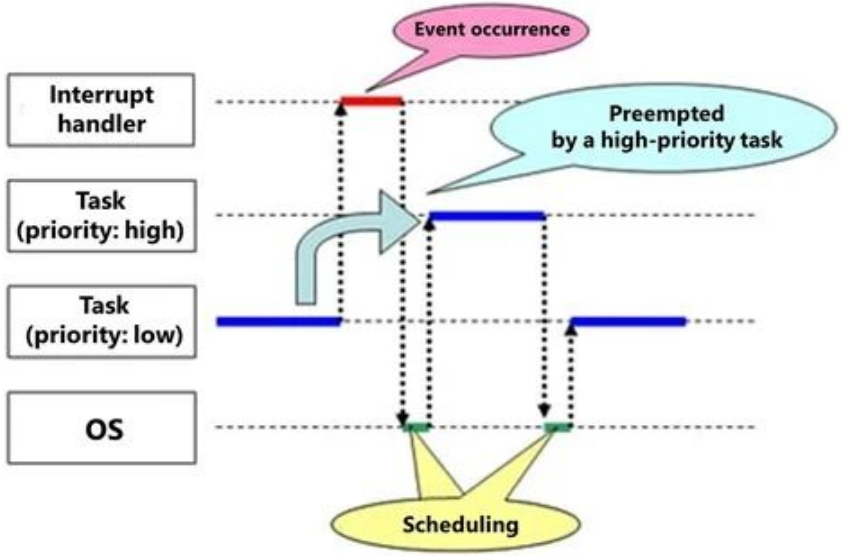
How it works: Highest priority runs first

Pros: ✅ Critical tasks first, ✅ Flexible

Cons: ❌ Starvation of low priority

Real-Time: Priority 1 → Priority 2 → Priority 3

Solution: Aging (gradually increase priority)

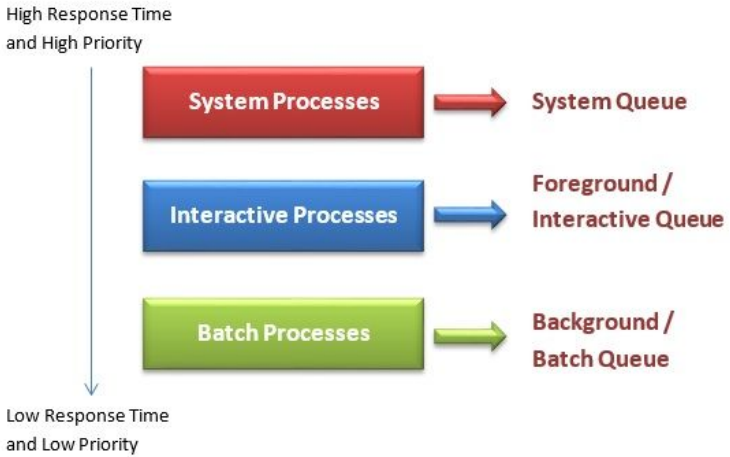


Multilevel Queue

Newton School of Technology

Multiple Queues with Different Rules:

- Queue 1: Real-time (FCFS, highest prio
- Queue 2: Interactive (RR, quantum=8m
- Queue 3: Batch (RR, quantum=16ms)



macOS uses 128 priority levels!

XNU Kernel Scheduler:

- Multilevel feedback queue
- 128 priority levels (0-127)
- Interactive processes get priority
- Time-sharing + real-time classes
- Responds to user interactions instantly

Scheduling Comparison

Newton School of Technology

Algorithm	Response	Fairness	Starvation	Complexity
FCFS	Poor	Fair	No	Low
SJF	Poor	Unfair	Yes	Low
RR	Excellent	Fair	No	Medium
Priority	Variable	Unfair	Yes	Medium
Multilevel	Good	Good	No	High

Context Switching Cost

Newton School of Technology

Every Switch Costs:

- Save/restore registers: ~200 cycles
- TLB flush: ~1000 cycles
- Cache misses: ~10,000+ cycles
- **Total:** 1-5 microseconds

Goal: Keep overhead < 1% of quantum

The Illusion of Parallelism

Newton School of Technology

How 4 cores run 100 processes:

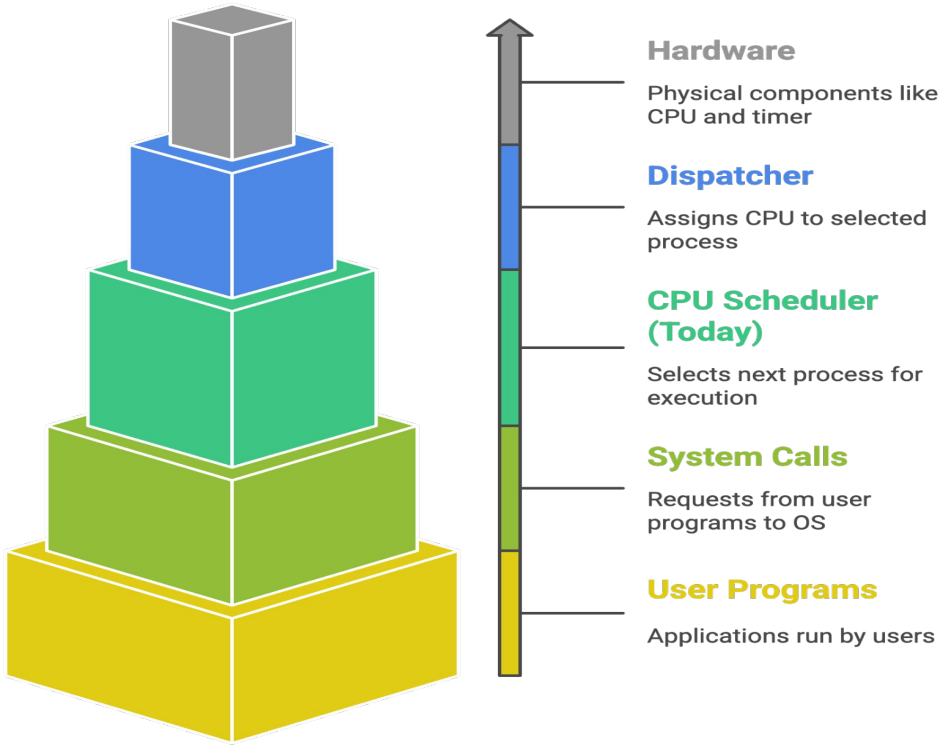
1. Round Robin gives everyone turns (every 10-100ms)
2. I/O-bound processes yield CPU quickly (1-10ms bursts)
3. Multilevel queues prioritize interactive apps
4. Context switching happens in microseconds
5. CPU+I/O overlap maximizes hardware use

You experience all 100 as "running simultaneously"

CPU Scheduling in the OS Stack

Newton School of Technology

Where we are:



Why This Matters

Newton School of Technology

Poor Scheduling =

- Frozen interfaces
- Dropped audio/video frames
- Slow app launches
- Poor battery life (mobile)

Good Scheduling =

- Smooth multitasking
- Instant app response
- Efficient resource use
- Better user experience

Key Takeaways

Newton School of Technology

- Scheduling is the **OS traffic controller**
- Different **goals** for different system types
- **Preemption** enables fair sharing (modern standard)
- **Process types** (CPU/I/O bound) affect decisions
- **No perfect algorithm** - all involve trade-offs
- **Your Mac** uses sophisticated multilevel scheduling

Thank You!

Newton School of Technology

Thank you for your attention. Feel free to ask questions or revisit the topics covered.